Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Guilherme Henrique Siqueira de Lana

Matrícula: 1446843

Outubro de 2026

1. Análise Inicial

Projeto: biblioteca `operacoes-mutante` (50 funções matemáticas em `src/operacoes.js`, 50 testes Jest). Ferramentas: Jest 29 e StrykerJS (`@stryker-mutator/core` + `jest-runner`).

Métrica	Valor inicial
Cobertura de linhas	98,64%
Cobertura de statements	85,41%
Cobertura de branches	58,82%
Mutation score	**73,71%** (154 mortos, 3 timeout, 44 sobreviventes, 12 sem cobertura, de 213 mutantes)

Discrepância: quase todas as linhas são executadas (98,64%), mas só 73,71% das alterações introduzidas são detectadas. A cobertura mede *o que foi executado*, não *o que foi verificado*. Cada função tem um único teste "caminho feliz", sem casos de borda (`a === b`, zero, negativos), sem ramos de erro com mensagem e sem arrays desordenados ou de tamanho par; a cobertura de branches (58,82%) já indicava isso.



Figura 1 – Relatório Stryker inicial: 73,71% de mutation score, 44 sobreviventes (vermelho) e 12 sem cobertura (laranja).

2. Análise de Mutantes Críticos

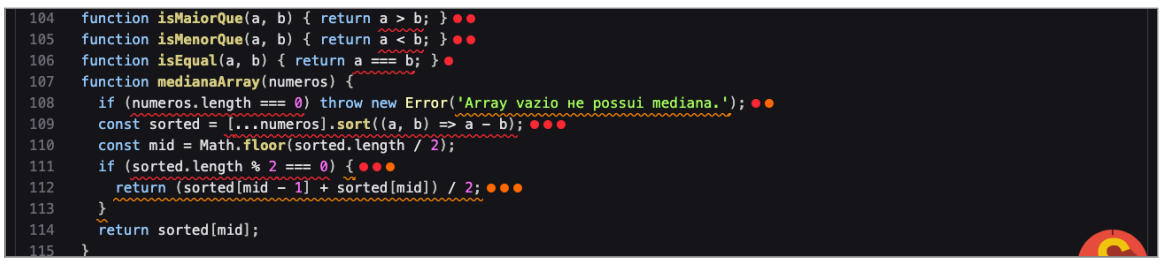


Figura 2 – Mutantes sobreviventes nas linhas 104–112 (isMaiorQue, isMenorQue, isEqual, medianaArray).

Mutante 1 – `isMaiorQue` (linha 104), EqualityOperator: `a > b` → `a >= b`. O único teste era `isMaiorQue(10, 5)`, que retorna `true` nas duas versões. Faltou o caso de fronteira `a === b`; o mesmo vale para `isMenorQue` (< → <=).

Mutante 2 – `medianaArray` (linha 109), MethodExpression: `[...numeros].sort((a,b)=>a-b)` → `[...numeros]` (sem ordenar). O teste usava `[1,2,3,4,5]`, que já está ordenado, então a ordenação nunca fazia diferença. Sobreviveram também as mutações do comparador (`a+b`, `() => undefined`).

Mutante 3 – `medianaArray` (linhas 111–112), NoCoverage/ConditionalExpression: o ramo de quantidade par de elementos (`(sorted[mid-1]+sorted[mid])/2`) nunca era executado, e `if (false)` também sobrevivia. Um bloco inteiro de lógica estava sem teste algum, apesar de 98% de cobertura de linhas.

3. Solução Implementada

Criei `test/operacoes.mutantes.test.js` (32 testes novos; total 82, todos passando com o código original). Principais casos e por que matam os mutantes:

```
expect(isMaiorQue(5, 5)).toBe(false);   // mata  >  → >=
expect(isMenorQue(5, 5)).toBe(false);   // mata  <  → <=
expect(isEqual(7, 8)).toBe(false);      // mata  return a===b → return true
expect(medianaArray([5, 1, 3])).toBe(3);        // exige ordenação correta
expect(medianaArray([10, 9, 1, 2, 5])).toBe(5); // comparador a-b (e não a+b)
expect(medianaArray([4, 1, 3, 2])).toBe(2.5);   // cobre o ramo par
expect(() => medianaArray([])).toThrow('possui mediana'); // mata if(false) e a string
```

Os testes são eficazes porque usam entradas em que o resultado do código original e o do mutante *diferem*: valores de fronteira, arrays desordenados, tamanhos par e ímpar, zero/negativos e asserção do texto das mensagens de erro (que mata mutantes de `StringLiteral`).

Mutantes equivalentes: após os novos testes restaram 7 mutantes que nenhum teste consegue matar, pois o comportamento é idêntico ao original: (a) `fatorial` sem o `if (n===0||n===1) return 1` (4 mutantes) – o laço já retorna 1; (b) `produtoArray([])` sem o `if` – o `reduce` com valor inicial 1 já retorna 1; (c) `clamp` com `valor <= min` / `valor >= max` – quando `valor === min` (ou max) retornar `min` ou `valor` é igual. Esses 7 foram marcados com `// Stryker disable next-line all` e justificativa no código; sem essa exclusão o score seria 96,71%.

4. Resultados Finais

Métrica	Inicial	Final
Testes	50	82
Mutantes mortos (+timeout)	157	195
Sobreviventes	44	0
Sem cobertura	12	0
Ignorados (equivalentes)	0	18*
Mutation score	73,71%	**100%** (96,71% sem excluir equivalentes)

*As 4 linhas marcadas geram 18 mutantes ignorados (7 deles os que sobreviviam mais os demais mutantes dessas linhas que já eram mortos).

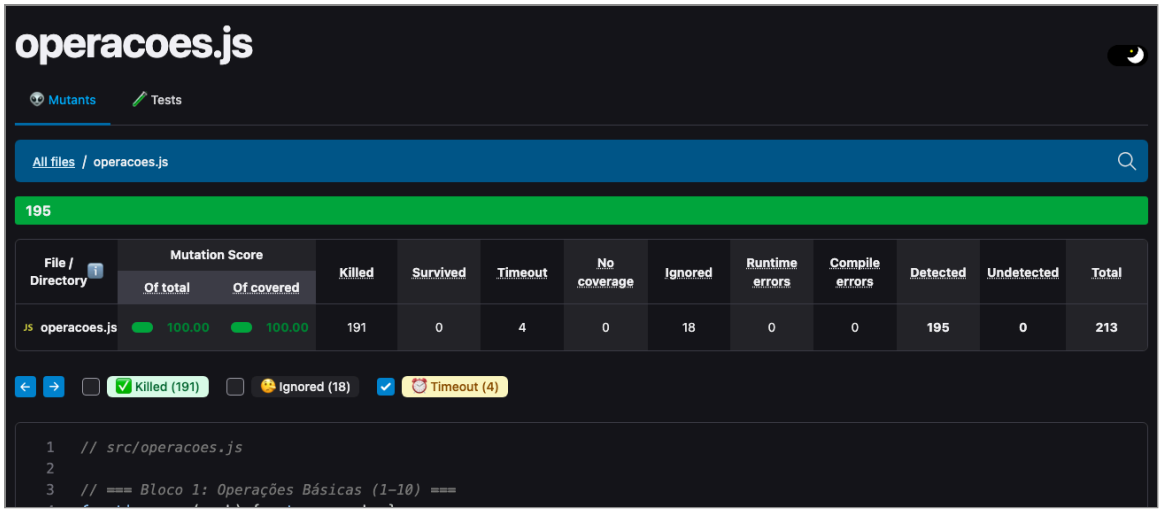


Figura 3 – Relatório Stryker final: 100% de mutation score, nenhum sobrevivente.

5. Conclusão

O experimento mostra que cobertura de código é uma métrica necessária, mas insuficiente: com 98,64% de cobertura de linhas, 26% das mutações passavam despercebidas. O teste de mutação mede a capacidade real da suíte de detectar defeitos, aponta exatamente quais asserções estão fracas (fronteiras, ramos não testados, mensagens de erro) e orienta a escrita de testes melhores. Também ensina a reconhecer mutantes equivalentes, que exigem julgamento humano, e a custo computacional razoável apenas em projetos pequenos como este.

Repositório (fork): https://github.com/gguilhermelana/operacoes-mutante